

## Schedule Future Jobs:

---

- The **at** and **Cron** modules are provided by Ansible are used to schedule future jobs on managed host.

## Ansible at Module

- The **at** module schedules **one-time tasks** on the target host.
- Perfect for delayed actions, maintenance windows, or post-deployment cleanup.

## Common Parameters

| Parameter      | Description                                        |
|----------------|----------------------------------------------------|
| <b>command</b> | The command/script to run at the scheduled time.   |
| <b>count</b>   | Number of units (e.g., 5 minutes, 2 hours).        |
| <b>units</b>   | Time units: minutes, hours, days, weeks.           |
| <b>state</b>   | present (default) or absent to delete a job.       |
| <b>unique</b>  | yes/no. Prevents duplicate jobs with same command. |
| <b>wait</b>    | Wait for job completion and return output.         |
| <b>user</b>    | Run the job as a specific user.                    |

## Examples:

### 1: Run a command once after 5 minutes

...

- name: Run a job after 5 minutes to check disk status

hosts: rhelnode

become: true

tasks:

- name: Copy disk\_check.sh to managed host

copy:

src: files/disk\_check.sh

dest: /usr/local/bin/disk\_check.sh

mode: '0755'

- name: Schedule disk check after 5 minutes

at:

command: "bash -lc '/usr/local/bin/disk\_check.sh'"

count: 5

units: minutes

unique: yes

...

### 2: Remove a scheduled at job

state: absent

## Ansible cron Module

- The **cron** module manages **recurring scheduled tasks**.
- Great for backups, log rotation, monitoring scripts, cleanup routines, etc.

## Common Parameters

| Parameter        | Description                                                 |
|------------------|-------------------------------------------------------------|
| <b>name</b>      | Identifier for the cron job (required).                     |
| <b>job</b>       | The command/script to run.                                  |
| <b>minute</b>    | Minute field (0–59, */5, etc.).                             |
| <b>hour</b>      | Hour field (0–23).                                          |
| <b>day</b>       | Day of month (1–31).                                        |
| <b>month</b>     | Month (1–12 or names).                                      |
| <b>weekday</b>   | Day of week (0–6 or names).                                 |
| <b>state</b>     | present (default) or absent.                                |
| <b>user</b>      | User whose crontab to modify.                               |
| <b>disabled</b>  | yes/no. Comment out the job without deleting it.            |
| <b>cron_file</b> | Write to a file under /etc/cron.d/ instead of user crontab. |
| <b>env</b>       | Set environment variable instead of a job.                  |
| <b>backup</b>    | Backup the crontab before modifying.                        |

### Example — Create a daily cron job

...

- name: Create a daily cron job

hosts: all

become: yes

tasks:

- name: Run backup every day at 3 AM

ansible.builtin.cron:

name: "Daily backup"

minute: "0"

hour: "3"

job: "/usr/local/bin/backup.sh"

...

### Example — Run every 15 minutes

...

- name: Run script every 15 minutes

hosts: all

become: yes

tasks:

- name: Monitor disk usage

ansible.builtin.cron:

name: "Disk monitor"

minute: "\*/15"

job: "/usr/local/bin/disk\_check.sh"

...

## Schedule at a specific time using cron\_file parameter

System administrators often need to run recurring jobs. It is best to run these jobs from system accounts rather than from user accounts. Schedule these jobs with system-wide crontab files `/etc/cron.d/` instead of with the `crontab` command. Job entries in the system-wide crontab files are similar to the users' `crontab` entries. The system-wide crontab files have an extra field `"user "` before the command field to specify the user that runs the command.

The `/etc/crontab` file and other files in the `/etc/cron.d/` system jobs. Always create custom crontab files in the recurring system jobs. Place the custom crontab file in `/etc/cron.d/` to prevent the package update from overwriting `/etc/crontab` file. Packages that require recurring system jobs place their crontab file in `/etc/cron.d/` directory with the job entries. Administrators also use this location to group related jobs into a single file.

The crontab system also includes repositories for scripts to run every hour, day, week, and month. These repositories are placed under the `/etc/cron.hourly`, `/etc/cron.weekly/` and `/etc/cron.monthly`, directories. These directories contain executable shell scripts, not crontab files

---

- name: Deploy cleanup script and manage cron job

hosts: all

become: true

tasks:

- name: Copy cleanup script to /usr/local/bin

ansible.builtin.copy:

src: cleanup.sh

dest: /usr/local/bin/cleanup.sh

mode: '0755'

owner: root

group: root

- name: Run cleanup script every day at 2:30 AM

ansible.builtin.cron:

name: "daily\_cleanup"

minute: "30"

hour: "2"

job: "/usr/local/bin/cleanup.sh"

cron\_file: "myapp\_cleanup"

user: root

...

## Remove a cron job

- name: Remove a cron job

hosts: all

become: yes

tasks:

- name: Delete disk monitor job

ansible.builtin.cron:

name: "Disk monitor"

state: absent

## Add cron job to a specific user

...

- name: Add cron job for user 'muhammad'

hosts: all

become: yes

tasks:

- name: User-specific cron

ansible.builtin.cron:

name: "User cleanup"

minute: "0"

hour: "1"

user: "muhammad"

job: "/home/muhammad/cleanup.sh"

✓ The script must be executable

✓ The script must be owned by the correct user

## EXAMPLE-PLAYBOOK

...

- name: Deploy cleanup and backup scripts, then schedule cron jobs

hosts: all

become: yes

tasks:

- name: Copy clean.sh to managed host

ansible.builtin.copy:

src: files/clean.sh

dest: /usr/local/bin/clean.sh

mode: '0755'

notify: restart\_cron

- name: Copy backup.sh to managed host

ansible.builtin.copy:

src: files/backup.sh

dest: /usr/local/bin/backup.sh

mode: '0755'

notify: restart\_cron

- name: Create nightly cleanup cron job

ansible.builtin.cron:

name: "Nightly cleanup"

minute: "0"

hour: "2"

job: "/usr/local/bin/clean.sh"

- name: Create daily backup cron job

ansible.builtin.cron:

name: "Daily backup"

minute: "0"

hour: "3"

```
job: "/usr/local/bin/backup.sh"
```

```
handlers:
```

```
- name: restart_cron
```

```
ansible.builtin.service:
```

```
name: crond
```

```
state: restarted
```

```
...
```

## Scripts:

- **1: Log and tmp cleanup script clean.sh** that:

- rotates logs
- clears temp files older than X days
- logs its own actions
- never deletes blindly

```
...
```

```
#!/bin/bash
```

```
LOGFILE="/var/log/cleanup.log"
```

```
TMP_DIR="/tmp"
```

```
DAYS=7
```

```
echo "==== Cleanup started at $(date) =====>> "$LOGFILE"
```

```
# Remove temp files older than 7 days
```

```
find "$TMP_DIR" -type f -mtime +$DAYS -print -delete >> "$LOGFILE" 2>&1
```

```
# Rotate system logs (safe, non-destructive)
```

```
if command -v logrotate >/dev/null 2>&1; then
```

```
    logrotate -f /etc/logrotate.conf >> "$LOGFILE" 2>&1
```

```
fi
```

```
echo "Cleanup completed at $(date)" >> "$LOGFILE"
```

```
echo "" >> "$LOGFILE"
```

```
...
```



## 2:Simple Backup Scrip backup.sh --> That:

- backs up a directory
- compresses it
- stores it with a timestamp
- logs everything
- avoids overwriting existing backups

```
```bash

#!/bin/bash

SOURCE_DIR="/etc"

BACKUP_DIR="/var/backups"

LOGFILE="/var/log/backup.log"

TIMESTAMP=$(date +"%Y%m%d_%H%M%S")

ARCHIVE="$BACKUP_DIR/etc_backup_${TIMESTAMP}.tar.gz"

mkdir -p "$BACKUP_DIR"

echo "==== Backup started at $(date) =====> "$LOGFILE"

# Create compressed archive

tar -czf "$ARCHIVE" "$SOURCE_DIR" >> "$LOGFILE" 2>&1

# Verify archive

if [ -f "$ARCHIVE" ]; then

    echo "Backup successful: $ARCHIVE" >> "$LOGFILE"

else

    echo "Backup FAILED at $(date)" >> "$LOGFILE"

fi

echo "Backup completed at $(date)" >> "$LOGFILE"

echo "" >> "$LOGFILE"

```
```

### 3: Disk\_check\_script. disk\_check.sh

```
```bash
```

```
#!/bin/bash
```

```
LOGFILE="/var/log/disk_check.log"
```

```
THRESHOLD=80
```

```
echo "==== Disk Check started at $(date) =====>> "$LOGFILE"
```

```
df -hP | grep -vE 'tmpfs|devtmpfs|squashfs|iso9660' | tail -n +2 | while read -r line; do
```

```
    MOUNT=$(echo "$line" | awk '{print $NF}')
```

```
    USAGE=$(echo "$line" | awk '{print $(NF-1)}' | tr -d '%')
```

```
    # Skip empty values
```

```
    if [[ -z "$USAGE" || -z "$MOUNT" ]]; then
```

```
        continue
```

```
    fi
```

```
    if [[ "$USAGE" -ge "$THRESHOLD" ]]; then
```

```
        echo "WARNING: $MOUNT is at ${USAGE}% usage" >> "$LOGFILE"
```

```
    else
```

```
        echo "OK: $MOUNT is at ${USAGE}% usage" >> "$LOGFILE"
```

```
    fi
```

```
done
```

```
echo "Disk Check completed at $(date)" >> "$LOGFILE"
```

```
echo "" >> "$LOGFILE"````
```

## The reboot Module

The reboot module safely restarts managed hosts and waits for them to come back online before continuing the play.

It's **safer** than using shell: reboot because it:

- Handles SSH reconnection automatically
- Waits until the host is reachable again
- Can set timeouts and delays
- Ensures idempotency (won't hang your playbook)

---

### Common Parameters

| Parameter         | Description                                              | Example                    |
|-------------------|----------------------------------------------------------|----------------------------|
| reboot_timeout    | Maximum seconds to wait for the host to come back online | 180                        |
| test_command      | Command used to verify host availability after reboot    | whoami                     |
| msg               | Custom message shown before reboot                       | "Rebooting after patching" |
| connect_timeout   | SSH connection timeout                                   | 5                          |
| pre_reboot_delay  | Seconds to wait before reboot                            | 10                         |
| post_reboot_delay | Seconds to wait after reboot before checking             | 15                         |

---

### Example Playbook: Safe Reboot After Patching

---

- name: Reboot servers safely after patching

hosts: webservers

become: yes

gather\_facts: no

tasks:

- name: Reboot after patching

ansible.builtin.reboot:

msg: "Rebooting after patch installation"

reboot\_timeout: 180

test\_command: whoami

- name: Verify system is back online

ansible.builtin.command: uptime

register: uptime\_result

- name: Display uptime

ansible.builtin.debug:

var: uptime\_result.stdout

---

### Quick Reboot Example

```
```yaml
```

- name: Force a quick reboot

ansible.builtin.reboot:

reboot\_timeout: 60

```
```
```

---

### Best Practice Tips

- Always use become: yes (reboot requires root privileges).
  - Combine with patching tasks (e.g., yum, apt) before reboot.
  - Use serial: in your play to reboot hosts one at a time.
  - Avoid shell: reboot — it won't wait for the host to return.
-